Finding Similar Items 1

Introduction to Data Mining

Seoul National University

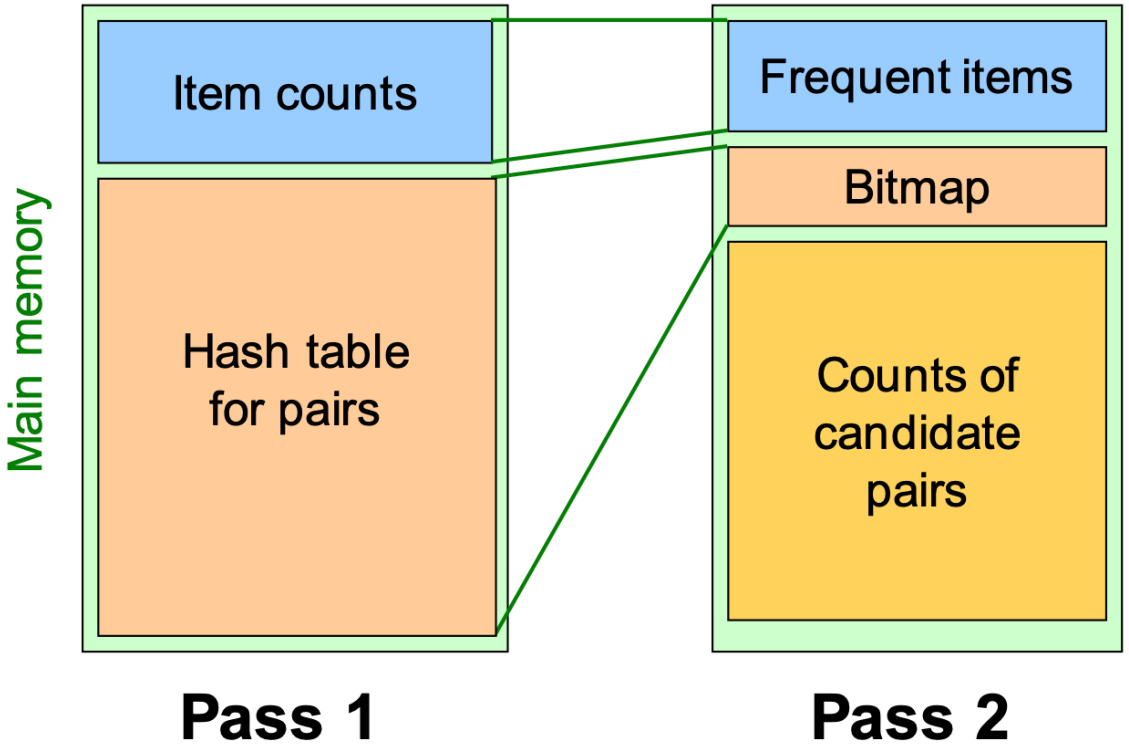
Spring 2026

Announcements

- **Assignments:**
  - HW1 will be posted today (deadline: April 9).

Recap: Frequent Itemsets

- A-Priori algorithm
- PCY algorithm and extensions
  - Multistage algorithm
  - Multihash algorithm
- Limited-pass algorithms
  - Random sampling
  - Toivonen's algorithm
  - SON algorithm



Outline

1. **<u>Finding Similar Items</u>**
2. Shingling
3. Minhashing
4. More on Minhashing

Finding Similar Items

- **Example:** Looking at web pages and finding near-duplicate pages.
- **Challenge:** What if $O(n^2)$ is infeasible due to too many pages?

The New York Times

Magnitude 6.8 Earthquake Shakes Chile

By The Associated Press

Aug. 1, 2019

SANTIAGO, Chile — A strong earthquake hit off the coast of central Chile on Thursday though there are no immediate reports of damage or injury.

The U.S. Geological Survey said the magnitude 6.8 quake struck at 2:28 p.m. local time (19:28 GMT) and was centered 59 miles (95 kilometers) northwest of San Antonio in the region of Valparaiso.

It was felt throughout the central part of the South American nation, but the national emergency office said there were no immediate reports of damage or injury and the navy discounted any threat of a tsunami.

AP

Magnitude 6.8 earthquake shakes Chile

Associated Press August 2, 2019

Click to copy

RELATED TOPICS

South America
General News
Latin America

SANTIAGO, Chile (AP) — A strong earthquake hit off the coast of central Chile on Thursday though there are no immediate reports of damage or injury.

The U.S. Geological Survey said the magnitude 6.8 quake struck at 2:28 p.m. local time (19:28 GMT) and was centered 59 miles (95 kilometers) northwest of San Antonio in the region of Valparaiso.

It was felt throughout the central part of the South American nation, but the national emergency office said there were no immediate reports of damage or injury and the navy discounted any threat of a tsunami.

Application 1: Finding Similar Documents

- **Example:** Finding textually similar documents in a large corpus.
  - In terms of character-level similarity instead of similar meaning.
  - E.g., web, news articles, etc.
- Many documents overlap significantly, but are not identical.
  - **Plagiarism:** Two homeworks overlap by 50%.
  - **Mirror sites:** Pages of mirror sites contain similar information.
  - **Similar articles:** Articles of “same story” in many newspapers.

Application 2: Collaborative Filtering

- **Example:** Recommend items liked by others with similar tastes.
- Online purchases (e.g., Amazon):
  - Two customers are similar if they have similar purchased items.
- Movie ratings (e.g., Netflix):
  - Similar to Amazon, but customers watch and rate movies or series.

Locality-Sensitivity

- Typical hash functions are random uniform mappings.
  - Let h be a hash function, and D_1 and D_2 be two documents.
  - $h(D_1)$ and $h(D_2)$ are independent of their actual similarity.
- What if we have the **locality-sensitive property?**
  - $P(h(D_1) = h(D_2))$ is large if D_1 and D_2 are similar.
- Hashing can be used for finding similar documents!

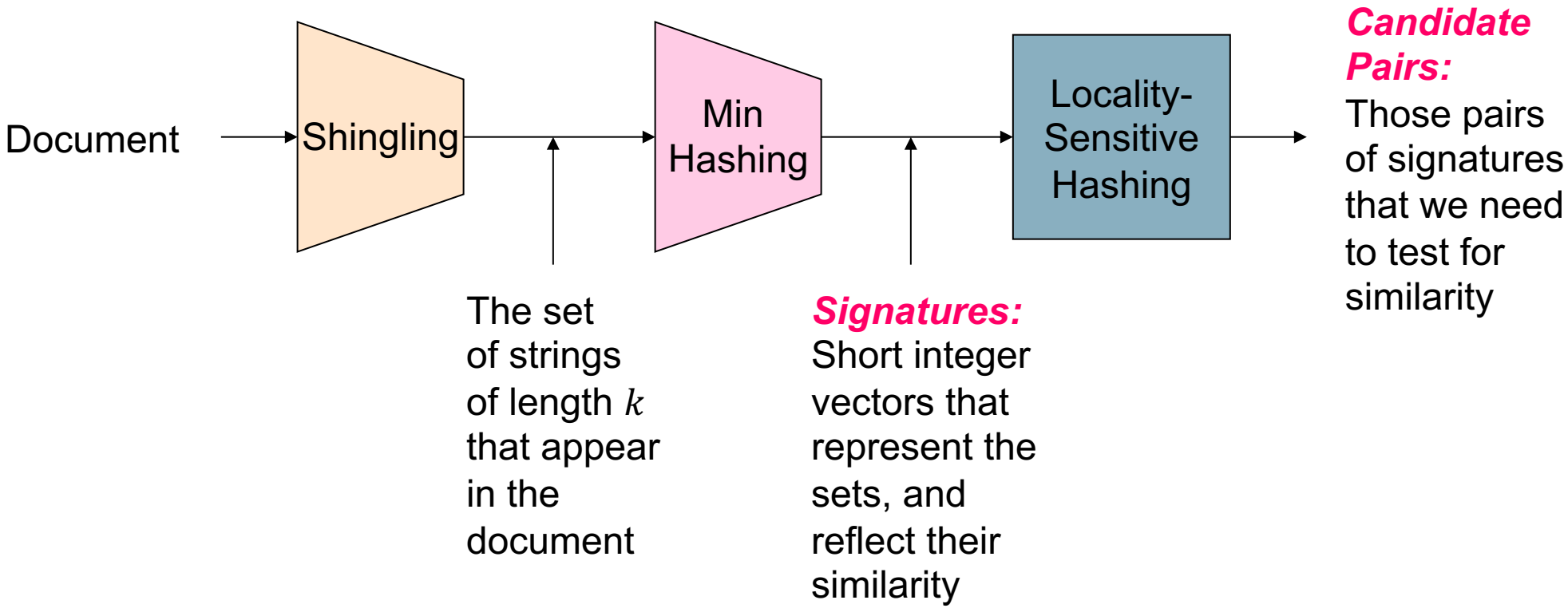
Locality-Sensitive Hashing

- **Locality-sensitive hashing (LSH):**
  - Finds pairs of similar items in a large set without the quadratic cost.
- **Basic idea:**
  - Throw items into buckets using several different hash functions.
  - Examine only the item pairs that share a bucket for at least one time.
- **Upside:** Only a small fraction of pairs are examined.
- **Downside:** False negatives – some pairs never get considered.

The Big Picture

1. **Shingling:**
   - Create a *characteristic matrix* that summarizes all documents.
2. **Minhashing:**
   - Create a *signature matrix* that reduces the size of the matrix.
   - It effectively approximates the distances between documents.
3. **Locality-sensitive hashing (in Part 2):**
   - Approximate all-pair comparisons in linear time.

The Big Picture: Illustration



Source: Stanford CS246 (2018)

Outline

1. Finding Similar Items
2. **<u>Shingling</u>**
3. Minhashing
4. More on Minhashing

Shingling of Documents

- ***k*-Shingles:** Represent a document as a set of length-k strings.
  - E.g., for "abcdabd" and $k = 2$, the 2-shingles are {ab, bc, cd, da, bd}.
  - Good for identifying lexically similar documents.
- Several options for handling white space (blank, tab, newline):
  - E.g., replace any sequence of whitespaces with a single blank.

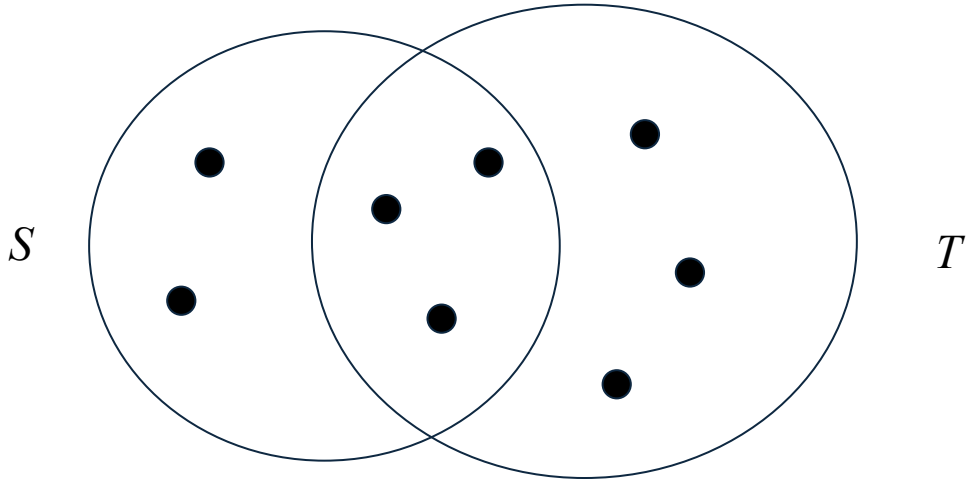
```
The   plane was ready
for touch down
```

$k=9$

```
{The plane, he plane , e plane w,  plane wa,
 plane was, lane was , ane was r, ne was re,
 ...}
```

Jaccard Similarity between Sets

- Similarity measure of sets based on relative size of intersection.
- **Jaccard similarity** between S and T is $|S \cap T| \,/\, |S \cup T|$.
- In the example below, $\mathrm{sim}(S, T) = 3/8$.



Shingles as Tokens

- **Idea:** Hash shingles to (say) 4-byte integers (called **tokens**).
  - Document is now a set of bucket indices of the shingles.
- Hashing 9-shingles to 4 bytes is better than using 4-shingles.
  - Most sequences of 4 characters are not found in typical documents.
    - # of distinct shingles $\ll 2^{32}$
  - If we hash down 9-shingles to 4 bytes, almost all 4 bytes are used.

Matrix Representation of Sets

- **Characteristic matrix:** Rows are elements, columns are sets.
  - 1 is in row e and column S if and only if $e \in S$.
  - Jaccard similarity can be computed by the column similarity.
  - **Warning:** Not the actual way how data is stored (due to sparsity).

Row (Token)	*Element*	S_1	S_2	S_3
0	"The plane"	1	0	0
1	"The cow j"	0	0	1
2	"Alice and"	0	1	0
3	"Roses are"	1	0	1

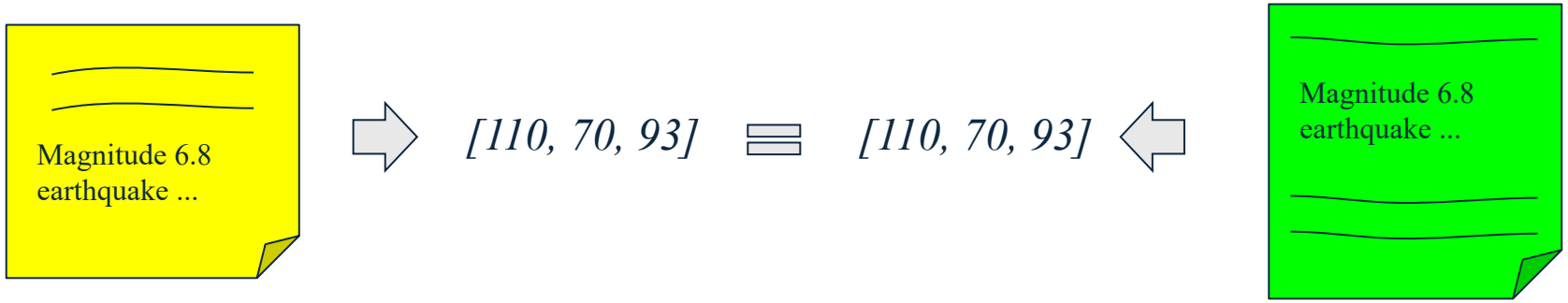
$\mathrm{sim}(S_1, S_3) = ?$

Outline

1. Finding Similar Items
2. Shingling
3. **<u>Minhashing</u>**
4. More on Minhashing

Similarity-Preserving Summaries of Sets

- For millions of documents, the characteristic matrix is too large.
- **Goal:** Replace each set by a **signature** (smaller representation).
  - Compare signatures of two sets to estimate the Jaccard similarity.
  - The larger the signatures, the more accurate the estimate.



Minhashing

- **Minhashing** creates signatures from the characteristic matrix.
  - Applies several random permutations of the rows.
  - For each permutation, the **minhash value** of each set (or column) is defined as the index of the first row that has 1.
- The result is a **signature matrix**.
  - Columns are sets, and rows are minhash values.
  - Normally much smaller than the characteristic matrix.

Example: Minhashing

1	0	1	1	0
2	0	0	1	1
3	1	0	0	0
4	0	1	0	1
5	0	0	0	1
6	1	1	0	0
7	0	0	1	0

Input Matrix

3	1	1	2

Signature Matrix

Source: Stanford CS246 (2022)

Example: Minhashing

7	1	0	1	1	0
6	2	0	0	1	1
5	3	1	0	0	0
4	4	0	1	0	1
3	5	0	0	0	1
2	6	1	1	0	0
1	7	0	0	1	0

Input Matrix

3	1	1	2
2	2	1	3

Signature Matrix

Source: Stanford CS246 (2022)

Example: Minhashing

6	7	1
3	6	2
1	5	3
7	4	4
2	3	5
5	2	6
4	1	7

0	1	1	0
0	0	1	1
1	0	0	0
0	1	0	1
0	0	0	1
1	1	0	0
0	0	1	0

Input Matrix

3	1	1	2
2	2	1	3
1	5	3	2

Signature Matrix

Source: Stanford CS246 (2022)

Subtle Point

- People ask whether the minhash value should be:
  - The original number of the row, or
  - The number in the permuted order (as we did in our example).
- **Answer:** It doesn't matter.
  - You only need to be consistent.
  - Assure that two columns get the same value if and only if
    - Their first 1's in the permuted order are in the same row.

Minhashing and Jaccard Similarity

- **Surprising property:**
  - Probability that minhash values are same = Jaccard similarity of sets.
- **Proof:**
  - Given columns S_1 and S_2, three classes of rows exist:
    - Type X: 1 in both columns.
    - Type Y: 1, 0 (or 0, 1) in columns.
    - Type Z: 0 in both columns.
  - If there are x rows of Type X and y rows of Type Y,
    - $\mathrm{sim}(S_1, S_2) = x/(x+y)$.
    - $\Pr(h(S_1) = h(S_2)) = \Pr(\text{meeting type X before type Y}) = x/(x+y)$.

Similarity of Signatures

- The **similarity of signatures** is the fraction of the same rows.
  - The **expected similarity of two signatures** equals
  - The **Jaccard similarity** of the sets that the signatures represent.
- The longer the signatures, the smaller will be the expected error.

Example: Similarity

Columns 1 & 2:
Jaccard similarity 1/4.
Signature similarity 1/3

Columns 2 & 3:
Jaccard similarity 1/5.
Signature similarity 1/3

Columns 3 & 4:
Jaccard similarity 1/5.
Signature similarity 0

0	1	1	0
0	0	1	1
1	0	0	0
0	1	0	1
0	0	0	1
1	1	0	0
0	0	1	0

Input Matrix

3	1	1	2
2	2	1	3
1	5	3	2

Signature Matrix

Source: Stanford CS246 (2022)

Outline

1. Finding Similar Items
2. Shingling
3. Minhashing
4. **<u>More on Minhashing</u>**

Implementation of Minhashing

- Explicit permutation is infeasible in a large characteristic matrix.
  - Suppose 1 billion rows.
- **Idea to approximate permutation:**
  - Apply a random **hash function** h to row indices $r = 1, \ldots, n$.
  - Sort the rows in order of their hash values $h(r)$.
  - The result is considered as random permutation.
- **Idea to implement:**
  - Create a "slot" matrix, and initialize all elements to ∞.
  - Update each element as we encounter smaller values.

Example: Implementation

Row	C1	C2
1	1	0
2	0	1
3	1	1
4	1	0
5	0	1

$h(x) = x \bmod 5$
$g(x) = (2x+1) \bmod 5$

	Sig1	Sig2
$h(1) = 1$	1	∞
$g(1) = 3$	3	∞
$h(2) = 2$	1	2
$g(2) = 0$	3	0
$h(3) = 3$	1	2
$g(3) = 2$	2	0
$h(4) = 4$	1	2
$g(4) = 4$	2	0
$h(5) = 0$	1	0
$g(5) = 1$	2	0

Source: Stanford CS246 (2022)

Algorithm

```
let T be a characteristic matrix
initialize a slot matrix M
for each hash function h_i and column c
    M(i, c) ← ∞
for each row r and column c
    if T(r, c) = 1 then
        for each hash function h_i
            if h_i(r) < M(i, c) then
                M(i, c) ← h_i(r)
```

Pop Quiz

- **Quiz:** Let's compute the signature matrix in this case.

Characteristic matrix

Row	S_1	S_2	S_3	g_1: $x + 1 \bmod 4$	g_2: $3x + 1 \bmod 4$
0	1	0	0	1	1
1	0	0	1	2	0
2	0	1	0	3	3
3	1	0	1	0	2

Signature matrix

Hash fn	S_1	S_2	S_3
g_1	∞	∞	∞
g_2	∞	∞	∞

Other Ways to Vectorize Documents

- **TF-IDF** is a popular way to create (a different type of) signatures.
  - Let M be the number of unique words in the corpus.
  - TF-IDF creates a length-M real-valued signature for each document.
- The i-th element of signature for document k is $\mathrm{TF}_{ik} \times \mathrm{IDF}_i$:
  - $\mathrm{TF}_{ik} = f_{ik} / \max_t f_{tk}$ where f_{tk} is the frequency of term t in document k.
  - $\mathrm{IDF}_i = \log(N/n_i)$ where term i appears in n_i of N documents.

Different Properties of Signatures

- The minhashed signatures are **integer vectors**.
  - We measure the similarity by # of the same integer elements.
- The TF-IDF signatures are **real-valued vectors**.
  - Two document signatures will not share any same elements.
  - We measure the *real-valued distance* between signatures.
  - Then, we need a different locality-sensitive hash function!

What are Good Signatures?

- Recently, **machine learning (ML)** is used to create signatures.
  - This topic is called **representation learning**.
  - These signatures are more commonly called **embeddings.**
  - All language models like ChatGPT work on embeddings.
- ML is powerful, but LLMs are even more powerful.
  - You may study natural language processing (NLP) more for this topic.

Summary

1. Finding Similar Items
2. Shingling
   - Jaccard similarity
   - Characteristic matrix
3. Minhashing
   - Signature matrix
   - Property of Minhashing
4. More on Minhashing
   - Simulating permutation